Advanced Artificial Intelligence and Machine Learning Lab 8

October 20, 2026

1 Lab 8

> Adapted from the UvA Deep Learning Tutorials' "Vision Transformers" notebook (`phlippe/uvadlc_notebooks`, Tutorial 15, MIT license). The source trains a larger ViT on the full CIFAR-10 training set for 180 epochs on a GPU; this lab uses a smaller ViT, a 4,000-image subset and 15 epochs so that it runs on a CPU within a lab session.

Session 10 argued that Vision Transformers lack the inductive biases (locality, translation equivariance) that convolutions have built in, and so need more data to reach the same performance a CNN gets "for free". This lab makes that argument concrete: we train a small ViT and a small CNN on the *exact same* CIFAR-10 subset, for the same number of epochs, and compare.

> **Instructor timing (2h session).** ~10 min: intro (this page) and setup - the CIFAR-10 download can be slow on a bad connection, so start this early. ~30 min: Exercise 1 - implement patch embedding and assemble the ViT. ~45 min: Exercise 2 - write the training and evaluation code, train the ViT and a CNN baseline under an identical, small data/epoch budget, and compare (training runs for several minutes - discuss the questions while it runs). ~25 min: Exercise 3 - extract and visualize the `[CLS]` token's attention in every layer. ~10 min buffer.

1.1 Setup

These labs are designed to run on Google Colab - no local Python install needed. Open this notebook via Colab's GitHub loader (`colab.research.google.com/github/KarikS/lecture_advanced_ai_ml/blob/main/<path-to-this-notebook`) or File > Open notebook > GitHub tab, repo `KarikS/lecture_advanced_ai_ml`, then run the cell below once per session. Running the notebook locally instead (e.g. via the `exercises/python/` venv described in that folder's README) works unchanged - the cell below is a no-op there. No GPU is needed - everything below is deliberately scaled to train on CPU. Exercise 2's two training runs take about 5 minutes on a 4-core laptop CPU, and can take noticeably longer on Colab's free tier.

[1]:
```
import sys

IN_COLAB = 'google.colab' in sys.modules

if IN_COLAB:
    import os
    if not os.path.exists('lecture_advanced_ai_ml'):
```

```
!git clone https://github.com/KarikS/lecture_advanced_ai_ml.git
```

1.2 Imports

[2]:
```
import time

import matplotlib.pyplot as plt
import torch
from matplotlib_inline.backend_inline import set_matplotlib_formats
from torch import nn, Tensor
from torch.utils.data import DataLoader, Subset
from torchvision.datasets import CIFAR10
from torchvision.transforms import Compose, Normalize, ToTensor

set_matplotlib_formats('png', 'pdf')
torch.manual_seed(0)

CIFAR_CLASSES = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',␣
↪'ship', 'truck']
CIFAR_MEAN = (0.4914, 0.4822, 0.4465)
CIFAR_STD = (0.2470, 0.2435, 0.2616)
```

1.3 Exercise 1: Patch Embedding and the ViT Architecture

Recall from Session 10: a ViT splits an image into fixed-size patches, linearly projects each patch into a token embedding, prepends a learnable `[CLS]` token, adds a learned position embedding to every token, and then runs an ordinary Transformer encoder (exactly the encoder from Session 7) over the resulting sequence. The final layer's `[CLS]` token representation is used for classification - the same mechanism BERT's `[CLS]` token uses (Session 8).

CIFAR-10 images are 32×32. With a patch size of 4, that gives $(32/4)^2 = 64$ patches per image, plus the `[CLS]` token: 65 tokens total, each `embed_dim`-dimensional - short enough sequences that even a CPU can process a batch quickly.

The cleanest way to implement patch embedding is a single strided convolution: a `Conv2d` with `kernel_size=stride=patch_size` applied to a $32 \times 32 \times 3$ image produces one `embed_dim`-dimensional vector per non-overlapping 4×4 patch, all in one call - exactly the linear patch projection from Session 10, just implemented via a convolution instead of an explicit reshape-then-linear (the two are mathematically identical here, since the "kernel" never slides over overlapping regions).

[3]:
```
class PatchEmbed(nn.Module):
    def __init__(self, patch_size: int, in_chans: int, embed_dim: int):
        super().__init__()
# TODO: Define a Conv2d that maps `in_chans` input channels to `embed_dim`
# output channels, with kernel_size and stride both equal to patch_size.

    def forward(self, x: Tensor) -> Tensor:
        x = self.proj(x)  # (batch, embed_dim, H / patch_size, W / patch_size)
```

```
# TODO: Flatten the two spatial dimensions into one sequence dimension, then
# move the embedding dimension last, giving shape (batch, num_patches, embed_dim).
# Hint: .flatten(2) then .transpose(1, 2).
```

[4]:
```
# Sanity check: a batch of 8 CIFAR-sized images should become 8 sequences of 64 patches.
patch_embed = PatchEmbed(patch_size=4, in_chans=3, embed_dim=32)
dummy_images = torch.randn(8, 3, 32, 32)
dummy_patches = patch_embed(dummy_images)
print(dummy_patches.shape)
assert dummy_patches.shape == (8, 64, 32)
```

Now the full ViT: prepend a learnable `[CLS]` token, add a learned position embedding, run PyTorch's built-in `nn.TransformerEncoder` (the same building block Session 7 derived from scratch), and classify from the final `[CLS]` representation.

[5]:
```
class ViT(nn.Module):
    def __init__(self, img_size: int = 32, patch_size: int = 4, embed_dim: int = 64,
                 depth: int = 4, heads: int = 4, mlp_dim: int = 128, num_classes: int = 10):
        super().__init__()
        self.patch_embed = PatchEmbed(patch_size, in_chans=3, embed_dim=embed_dim)
        num_patches = (img_size // patch_size) ** 2

# TODO: Create the learnable [CLS] token, shape (1, 1, embed_dim), and the
# learned position embedding, shape (1, num_patches + 1, embed_dim). Both
# should be nn.Parameter so they are optimized during training.
        nn.init.trunc_normal_(self.pos_embed, std=0.02)

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=embed_dim, nhead=heads, dim_feedforward=mlp_dim,
            dropout=0.1, batch_first=True, activation='gelu',
        )
        self.encoder = nn.TransformerEncoder(encoder_layer, num_layers=depth)
        self.norm = nn.LayerNorm(embed_dim)
        self.head = nn.Linear(embed_dim, num_classes)

    def forward(self, x: Tensor) -> Tensor:
        batch_size = x.size(0)
        x = self.patch_embed(x)

# TODO: Expand cls_token to this batch size and concatenate it in front of
# the patch tokens (along the sequence dimension), then add the position
```

```
# embedding.

        x = self.encoder(x)
# TODO: Take the first token (the [CLS] token) from the encoder output,
# normalize it, and pass it through the classification head.
```

[6]:
```
vit_sanity = ViT()
dummy_logits = vit_sanity(dummy_images)
print(dummy_logits.shape)
assert dummy_logits.shape == (8, 10)
```

1.4 Exercise 2: ViT vs. CNN, Same Data, Same Budget

We now load CIFAR-10. Full CIFAR-10 is 50,000 training images - more than we can afford several training runs over on CPU within a lab session. We deliberately train on a *small* subset (a few thousand images) for a *few* epochs - this is the whole point: we want to see how each architecture behaves when data and training time are both scarce, not to reach the source tutorial's 180-epoch, full-dataset accuracy.

[7]:
```
transform = Compose([ToTensor(), Normalize(CIFAR_MEAN, CIFAR_STD)])
train_full = CIFAR10(root='.data', train=True, download=True,␣
 ↪transform=transform)
test_full = CIFAR10(root='.data', train=False, download=True,␣
 ↪transform=transform)

N_TRAIN = 4000
N_TEST = 1000

# Shuffle before subsetting: some datasets store examples grouped by class.␣
 ↪CIFAR-10
# does not, but it is good practice never to assume a raw ordering is safe to␣
 ↪slice.
generator = torch.Generator().manual_seed(0)
train_subset_idx = torch.randperm(len(train_full), generator=generator)[:N_TRAIN]
test_subset_idx = torch.randperm(len(test_full), generator=generator)[:N_TEST]

train_dataset = Subset(train_full, train_subset_idx.tolist())
test_dataset = Subset(test_full, test_subset_idx.tolist())

train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True,␣
 ↪num_workers=2)
test_loader = DataLoader(test_dataset, batch_size=256, num_workers=2)
# The same training images again, unshuffled - only used to measure training␣
 ↪accuracy.
train_eval_loader = DataLoader(train_dataset, batch_size=256, num_workers=2)
```

Here is a small CNN baseline - three convolution/pooling blocks followed by two linear layers, the same recipe Lab 3 used for MNIST, just scaled up slightly for CIFAR-10's larger, 3-channel images.

[8]:
```
class SmallCNN(nn.Module):
    def __init__(self, num_classes: int = 10):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1), nn.ReLU(), nn.
 ↪MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.ReLU(), nn.
 ↪MaxPool2d(2),
            nn.Conv2d(64, 64, kernel_size=3, padding=1), nn.ReLU(), nn.
 ↪MaxPool2d(2),
            nn.Flatten(),
            nn.Linear(64 * 4 * 4, 128), nn.ReLU(),
            nn.Linear(128, num_classes),
        )

    def forward(self, x: Tensor) -> Tensor:
        return self.net(x)
```

[9]:
```
def accuracy(model: nn.Module, loader: DataLoader) -> float:
    model.eval()
    n_correct, n_total = 0, 0
    with torch.no_grad():
        for x, y in loader:
# TODO: Count how many images in this batch are classified correctly
# (predicted class = argmax over the logits) and add that to n_correct;
# add the batch size to n_total.
    return n_correct / n_total


def train_and_evaluate(model: nn.Module, epochs: int, lr: float = 1e-3) ->␣
 ↪list[float]:
    optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=1e-4)
    loss_fn = nn.CrossEntropyLoss()
    test_accuracies = []

    t0 = time.time()
    for epoch in range(1, epochs + 1):
        model.train()
        total_loss, n = 0.0, 0
        for x, y in train_loader:
# TODO: One optimization step on this batch: forward pass, cross-entropy
# loss with loss_fn, backward pass, optimizer step. Call the loss `loss`.
            total_loss += float(loss) * len(y)
            n += len(y)
```

```
        test_acc = accuracy(model, test_loader)
        test_accuracies.append(test_acc)
        print('EPOCH:\t{:2}\tTRAIN LOSS:\t{:.3f}\tTEST ACCURACY:\t{:.
 ↪3f}\tELAPSED:\t{:.1f}s'
              .format(epoch, total_loss / n, test_acc, time.time() - t0))
    return test_accuracies

# Both models get the same data, epochs, optimizer and learning rate. 1e-3 was␣
 ↪the better
# of the two learning rates tested (3e-4, 1e-3) for *both* models under this␣
 ↪budget.
EPOCHS = 15

print('Training CNN baseline...')
torch.manual_seed(0)
cnn_model = SmallCNN()
cnn_accuracies = train_and_evaluate(cnn_model, epochs=EPOCHS)

print('Training ViT...')
torch.manual_seed(0)
vit_model = ViT()
vit_accuracies = train_and_evaluate(vit_model, epochs=EPOCHS)
```

[10]:
```
plt.plot(range(1, EPOCHS + 1), cnn_accuracies, marker='o', label='CNN')
plt.plot(range(1, EPOCHS + 1), vit_accuracies, marker='o', label='ViT')
plt.xlabel('epoch')
plt.ylabel('test accuracy')
plt.legend()
plt.show()

for name, model in [('CNN', cnn_model), ('ViT', vit_model)]:
    print('{}: train accuracy {:.3f}, test accuracy {:.3f}'
          .format(name, accuracy(model, train_eval_loader), accuracy(model,␣
 ↪test_loader)))
```

On this small a data/epoch budget, the CNN should end up clearly ahead of the ViT on test accuracy - when this lab was tested over three random seeds, the CNN reached 0.54-0.58 and the ViT 0.44-0.46. The CNN's convolutional inductive bias (locality, translation equivariance) gives it a head start the ViT has to *learn* from data, exactly as Session 10 argued. The source tutorial shows how much it takes to close that gap from scratch: with the full 50,000-image training set and 180 epochs on a GPU, its ViT reaches 75.6% test accuracy - while the CNNs in the same tutorial series reach around 90% on CIFAR-10.

Question: Now look at the *training* accuracies printed above, not just the test accuracies. Is the

ViT behind because it overfits the small training set?

Question: If you had a much bigger compute and data budget, would you expect this gap to shrink, stay the same, or reverse?

1.5 Exercise 3: What Does the [CLS] Token Actually Attend To?

The ViT's final classification relies entirely on the [CLS] token's representation after the last encoder layer. We can inspect, in each of the 4 encoder layers, the attention weights from the [CLS] token to every patch, to see where it collects its information from.

`nn.TransformerEncoderLayer` doesn't expose its attention weights by default - we need to call its internal `self_attn` module directly with `need_weights=True`. Our encoder layers apply attention to their input *before* any normalization (PyTorch's default, post-norm layout), so the input to layer l's `self_attn` is simply the output of layer $l-1$.

[11]:
```
def get_cls_attention(model: ViT, x: Tensor, layer: int) -> Tensor:
    """Attention from [CLS] to the 64 patches in encoder layer `layer`, shape
 ↪(batch, 64)."""
    model.eval()
    with torch.no_grad():
        layers = model.encoder.layers
# TODO:
# 1. Rebuild the encoder's input exactly as ViT.forward does: patch embedding,
# [CLS] token in front, plus the position embedding.
# 2. Run it through layers[0], ..., layers[layer - 1] to get the input of
# layers[layer].
# 3. Call layers[layer].self_attn(h, h, h, need_weights=True,
# average_attn_weights=True). It returns (output, attention weights); the
# weights have shape (batch, seq_len, seq_len), averaged over the heads.
# 4. Row 0 is the [CLS] token's attention to every token. Keep only its
# attention to the 64 patches (drop column 0, the [CLS] token itself).
    return cls_to_patches

N_SAMPLES = 4
sample_images = torch.stack([test_dataset[i][0] for i in range(N_SAMPLES)])
sample_labels = [CIFAR_CLASSES[test_dataset[i][1]] for i in range(N_SAMPLES)]
n_layers = len(vit_model.encoder.layers)
cls_attention = torch.stack([get_cls_attention(vit_model, sample_images, l) for
 ↪l in range(n_layers)])
print(cls_attention.shape)  # (layers, images, 64) -- one attention weight per
 ↪patch
```

Before looking at pictures, two numbers per layer. If the [CLS] token attended to all 64 patches equally, every weight would be $1/64$ and the entropy of each attention row would be $\log 64 \approx 4.16$. We print how far the largest weight is above that uniform value, and the average entropy:

[12]:
```
for l in range(n_layers):
    a = cls_attention[l]
    entropy = -(a / a.sum(1, keepdim=True) * (a / a.sum(1, keepdim=True)).log()).
 ↪sum(1).mean()
    print('layer {}: largest weight = {:.1f} x uniform, entropy = {:.2f}␣
 ↪(uniform: 4.16)'
          .format(l, float((a.max(1).values * 64).mean()), float(entropy)))
```

[13]:
```
def unnormalize(img: Tensor) -> Tensor:
    mean = torch.tensor(CIFAR_MEAN).view(3, 1, 1)
    std = torch.tensor(CIFAR_STD).view(3, 1, 1)
    return (img * std + mean).clamp(0, 1)

# One shared color scale for every map - otherwise each map would be stretched␣
 ↪to its
# own min/max and even a perfectly flat map would look like it had strong␣
 ↪structure.
vmax = float(cls_attention.max())
fig, axes = plt.subplots(N_SAMPLES, n_layers + 1, figsize=(2.4 * (n_layers + 1),␣
 ↪2.4 * N_SAMPLES))
for i in range(N_SAMPLES):
    axes[i, 0].imshow(unnormalize(sample_images[i]).permute(1, 2, 0))
    axes[i, 0].set_title(sample_labels[i])
    for l in range(n_layers):
        im = axes[i, l + 1].imshow(cls_attention[l, i].view(8, 8),␣
 ↪cmap='viridis', vmin=0, vmax=vmax)
        axes[i, l + 1].set_title(f'layer {l}')
    for ax in axes[i]:
        ax.axis('off')
fig.colorbar(im, ax=axes, shrink=0.6, label='attention weight')
plt.show()
```

The first layer should look almost perfectly flat - and almost the same for every image. In the later layers, the `[CLS]` token concentrates on a few patches (when this lab was tested, the largest weight was roughly 7-12 times the uniform value) and which patches it picks differs from image to image. Whether those patches are actually "on the object" is worth checking with your own eyes rather than assuming - with this little training, often they won't be.

Question: Why is the first layer's `[CLS]` attention almost the same for every image?

1.6 Conclusion

Here's what you should take away from this lab:

- Patch embedding (Exercise 1) is genuinely simple: a single strided convolution turns an image into a sequence of tokens, and everything after that is the same Transformer encoder Session

7 already derived - "a Transformer for images" really is that direct.

- The CNN-vs-ViT gap under a small data/epoch budget (Exercise 2) is the measurable version of the inductive-bias argument Session 10 made in words: convolutions build in assumptions about images that a ViT has to learn from data instead. Here that shows up twice - the ViT fits the training data more slowly, *and* it transfers less of what it has learned to new images.
- Attention weights (Exercise 3) are directly inspectable: you can ask "what is this token looking at?" and get a literal answer. But read them carefully - use one shared color scale, check them against the uniform baseline, and compare layers, before concluding that a model "looks at" anything.